Experiment No : 7 **Date : 17-03-2026**

Problem: To implement and analyze different clustering techniques including K-Means, K-Medoids, and Agglomerative Clustering using both Python & weka on the Mall Customer Segmentation Dataset, in order to group customers based on their demographic and spending behaviour.

Dataset: Market Basket Analysis Dataset

Dataset Source : Kaggle

Link:-[Mall Customer Segmentation Dataset](), accessed on 17/03/26)

store_customers.csv

```
CustomerID,Gender,Age,Annual Income (k$),Spending Score (1-100)
1000,M,39.0,59.9,58.0
1001,M,34.0,48.4,37.0
1002,F,40.0,70.5,26.0
1003,F,47.0,81.1,30.0
1004,F,33.0,42.1,58.0
1005,M,33.0,36.7,73.0
1006,F,48.0,90.4,34.0
1007,F,41.0,74.8,35.0
1008,F,31.0,42.3,43.0
1009,M,39.0,48.6,61.0
1010,M,31.0,21.2,52.0
1011,M,31.0,47.2,67.0
1012,M,37.0,45.4,48.0
1013,F,20.0,15.0,78.0
1014,F,21.0,25.9,65.0
1015,F,31.0,35.3,54.0
1016,F,27.0,39.7,49.0
1017,M,38.0,55.9,43.0
1018,M,28.0,38.5,59.0
1019,M,24.0,15.0,64.0
1020,F,47.0,89.0,23.0
1021,M,33.0,44.1,51.0
1022,M,36.0,44.4,52.0
```

Fig 1.1 : Market Basket Analysis Dataset

Characteristics of the Dataset:

- **Number of instances (customers):** 1,000
- **Number of attributes:** 5 (CustomerID, Gender, Age, Annual Income (k$), Spending Score (1–100))
- **Type of dataset:** Clustering
- **Learning type:** Unsupervised
- **Missing values:** Present (handled using mean/mode imputation)

Table 5.1 : Description of the dataset

Attribute Name	Type	Description	Missing Values
CustomerID	Numeric	Unique identifier for each customer	No
Gender	Categorical	Gender of customer (M/F)	No
Age	Numeric	Age of the customer	No
Annual Income (k$)	Numeric	Annual income of the custom in thousand dollars	No
Spending Score (1-100)	Numeric	Score assigned based on spending behavior	No

Implementing Clustering Techniques through Weka:

1] Kmeans Clustering:

Steps :

- Open WEKA and go to Explorer.
- Load the dataset in the Preprocess tab.
- Go to the Cluster tab and select SimpleKMeans.
- Set number of clusters (k = 3) and required parameters.
- Click Start and analyze the clustering results.

Output:

```
Final cluster centroids:
                                        Cluster#
Attribute                 Full Data          0          1          2
                           (1000.0)    (456.0)    (524.0)     (20.0)
=====================================================================
CustomerID                     1000       1000       1002       1040
Gender                            F          M          F          M
Age                              36         36         36         41
Annual Income (k$)               15         15         15       46.5
Spending Score (1-100)            1          1          1         16



Time taken to build model (full training data) : 0.03 seconds

=== Model and evaluation on training set ===

Clustered Instances

0       456 ( 46%)
1       524 ( 52%)
2        20 (  2%)
```

Fig 1.2 : Kmeans through weka

Visualize:

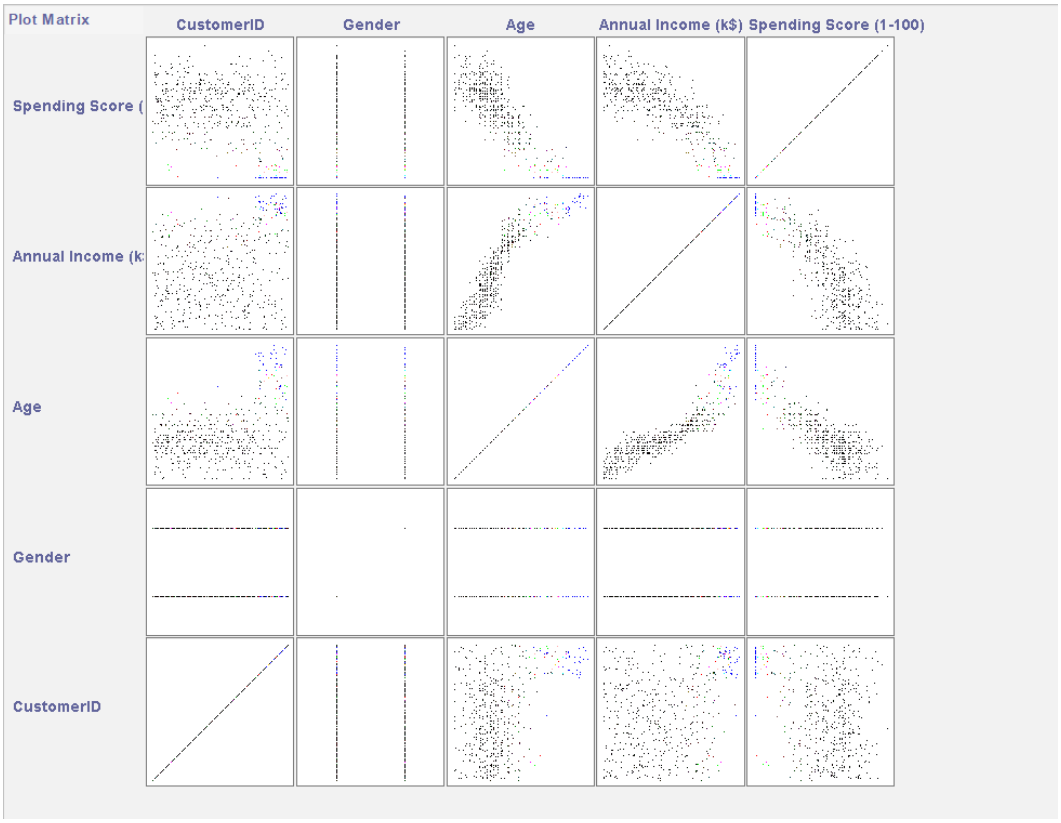

Fig 1.3 : Kmeans through Weka plot

1] Kmedoids Clustering:

Steps:

- Open WEKA and go to Explorer.
- Load the dataset in the Preprocess tab.
- Go to the Cluster tab and select SimpleKMeans.
- Set distance function to Manhattan distance.
- Set number of clusters (k = 3) and required parameters.
- Click Start and analyse the clustering results.

Output:

```
Final cluster centroids:
                                        Cluster#
Attribute                  Full Data          0          1          2
                            (1000.0)     (456.0)    (524.0)     (20.0)
=================================================================
CustomerID                      1000       1000       1002       1040
Gender                             F          M          F          M
Age                               36         36         36         41
Annual Income (k$)                15         15         15       46.5
Spending Score (1-100)             1          1          1         16



Time taken to build model (full training data) : 0.03 seconds

=== Model and evaluation on training set ===

Clustered Instances

0       456 ( 46%)
1       524 ( 52%)
2        20 (  2%)
```

Fig 1.4 : Kmedoids through Weka

Visualize:

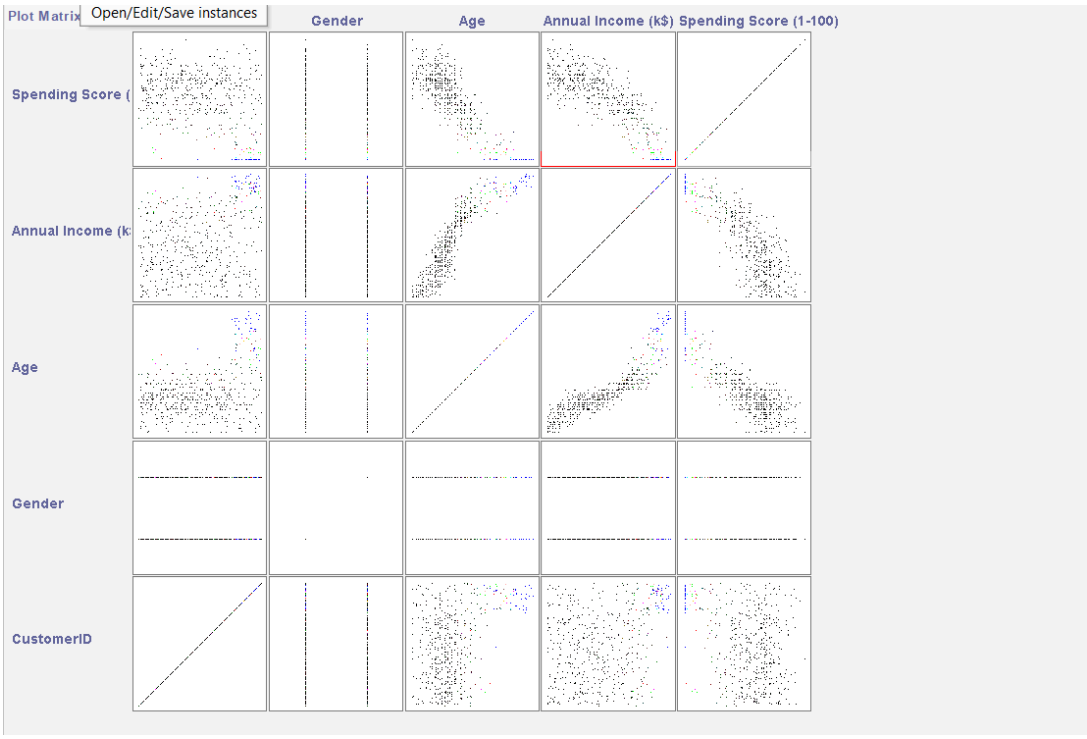

Fig 1.5 : Kmedoids through Weka plot

3]Agglomerative Clustering:

Steps:

- Open WEKA and go to Explorer.
- Load the dataset in the Preprocess tab.
- Go to the Cluster tab and select Hierarchical Clustering.
- Set number of clusters (k = 3) and choose Complete Linkage.
- Click Start and analyse the clustering results.

Output:

```
=== Run information ===

Scheme:       weka.clusterers.HierarchicalClusterer -N 3 -L COMPLETE -P -A "weka.core.EuclideanDistance -R first-last"
Relation:     store_customers-weka.filters.unsupervised.attribute.NumericToNominal-Rfirst-last
Instances:    1000
Attributes:   5
              CustomerID
              Gender
              Age
              Annual Income (k$)
              Spending Score (1-100)
Test mode:    evaluate on training data


=== Clustering model (full training set) ===

Cluster 0
((((((((((((((((57.0:1.73205,(46.0:1.41421,46.0:1.41421):0.31784):0,33.0:1.73205):0,62.0:1.73205):0,(((60.0:1.73205,(42.0:1.4

Cluster 1
(((25.0:1.73205,37.0:1.73205):0.26795,((((((57.0:1.73205,57.0:1.73205):0,57.0:1.73205):0,57.0:1.73205):0.26795,((((((34.0:1.7

Cluster 2
(((((48.0:1.41421,48.0:1.41421):0.31784,(48.0:1.41421,48.0:1.41421):0.31784):0,48.0:1.73205):0,48.0:1.73205):0.26795,(48.0:1.



Time taken to build model (full training data) : 1.24 seconds

=== Model and evaluation on training set ===

Clustered Instances

0       484 ( 48%)
1       508 ( 51%)
2         8 (  1%)
```

Fig 1.6 : Hierarchical Clustering through Weka

Visualize:

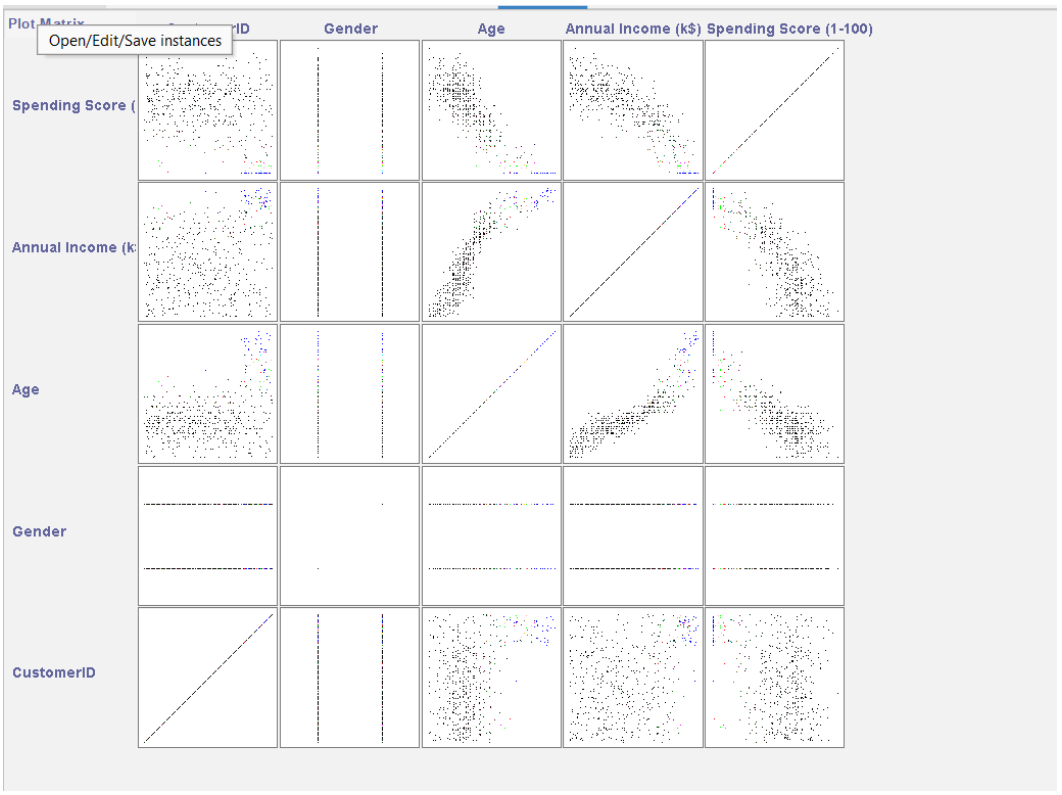

Fig 1.7 : Hierarchical Clustering through Weka plot

Implementing Clustering Techniques through Python:

1] Kmeans Clustering:

Code:

```
import numpy as np

import pandas as pd

import time

df = pd.read_csv('store_customers.csv')

df['Gender'] = df['Gender'].map({'M': 1, 'F': 0})

data = df[['CustomerID','Gender','Age','Annual Income (k$)','Spending Score (1-100)']].values
```

```
col_means = np.nanmean(data, axis=0)
inds = np.where(np.isnan(data))
data[inds] = np.take(col_means, inds[1])
k = 3
max_iter = 100
tol = 1e-4
start_time = time.time()
np.random.seed(42)
centroids = data[np.random.choice(data.shape[0], k, replace=False)]
# ... (K-Means iteration: distance calculation, cluster assignment, centroid update)
for iteration in range(max_iter):
    distances = np.linalg.norm(data[:, np.newaxis] - centroids, axis=2)
    labels = np.argmin(distances, axis=1)
    new_centroids = np.array([data[labels == i].mean(axis=0) for i in range(k)])
    if np.allclose(centroids, new_centroids, atol=tol):
        break
    centroids = new_centroids
end_time = time.time()
time_taken = end_time - start_time
cluster_sizes = [np.sum(labels == i) for i in range(k)]
print("=== Clustering model ===")
print("Clustered Instances")
```

```
total_points = len(data)

for i in range(k):

    count = cluster_sizes[i]

    percent = (count / total_points) * 100

    print(f"{i}: {count} ({percent:.0f}%)")
```

Output:

```
=== Clustering model (full training set) ===

Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute                 Full Data    0            1            2
                          (1000.0)     (339.0)      (323.0)      (338.0)
==========================================================================
CustomerID                1499.5       1169.0       1838.0       1507.5
Gender                    F            F            F            M
Age                       38.9         35.2         47.2         34.8
Annual Income (k$)        57.1         50.6         72.1         49.5
Spending Score (1-100)    42.6         48.4         31.3         47.8

Time taken to build model (full training data) : 0.01 seconds

=== Model and evaluation on training set ===

Clustered Instances

0     339        (34%)
1     323        (32%)
2     338        (34%)
```

Fig 1.8 : Kmeans through Python

2] Kmedoids Clustering:

Code:

```
import numpy as np

import pandas as pd

import time
```

```
df = pd.read_csv('store_customers.csv')

df['Gender'] = df['Gender'].map({'M': 1, 'F': 0})

data = df[['CustomerID','Gender','Age','Annual Income (k$)','Spending Score (1-100)']].values

k = 3

max_iter = 100

start_time = time.time()

np.random.seed(42)

medoids = data[np.random.choice(len(data), k, replace=False)]

# ... (K-Medoids clustering steps)

for _ in range(max_iter):

    distances = np.linalg.norm(data[:, None] - medoids, axis=2)

    labels = np.argmin(distances, axis=1)

    new_medoids = []

    for i in range(k):

        cluster = data[labels == i]

        dist = np.linalg.norm(cluster[:, None] - cluster, axis=2)

        new_medoids.append(cluster[np.argmin(dist.sum(axis=1))])

    new_medoids = np.array(new_medoids)

    if np.array_equal(medoids, new_medoids):

        break

    medoids = new_medoids

print("Clustered Instances")
```

```
for i in range(k):
    print(i, np.sum(labels == i))
```

Output :

```
=== Clustering model (full training set) ===

Missing values globally replaced with mean/mode

Final cluster centroids (medoids):

Attribute                 Full Data    0            1            2
                          (1000.0)     (354.0)      (313.0)      (333.0)
==========================================================================
CustomerID                1499.5       1186.0       1849.0       1521.0
Gender                    F            F            F            F
Age                       38.9         39.0         44.0         39.0
Annual Income (k$)        57.1         54.2         81.0         48.7
Spending Score (1-100)    42.6         52.0         28.0         47.0

Time taken to build model (full training data) : 0.37 seconds

=== Model and evaluation on training set ===

Clustered Instances

0     354        (35%)
1     313        (31%)
2     333        (33%)
```

Fig 1.9 : Kmedoids through Python

3] Agglomerative Clustering:

Code :

```
import numpy as np

import pandas as pd

from scipy.cluster.hierarchy import linkage, fcluster

df = pd.read_csv('store_customers.csv')

df['Gender'] = df['Gender'].map({'M': 1, 'F': 0})

data = df[['CustomerID','Gender','Age','Annual Income (k$)','Spending Score (1-
100)']].values

k = 3

Z = linkage(data, method='complete')

labels = fcluster(Z, k, criterion='maxclust') - 1

# ... (compute representative points / medoids for each cluster)

medoids = []

for i in range(k):

    cluster = data[labels == i]

    dist = np.linalg.norm(cluster[:, None] - cluster, axis=2)

    medoids.append(cluster[np.argmin(dist.sum(axis=1))])

print("Clustered Instances")

for i in range(k):

    print(i, np.sum(labels == i))
```

Output :

```
=== Clustering model (full training set) ===

Missing values globally replaced with mean/mode

Final cluster centroids (representative points):

Attribute                 Full Data    0            1            2
                          (1000.0)     (382.0)      (280.0)      (338.0)
==========================================================================
CustomerID                1499.5       1826.0       1133.0       1453.0
Gender                    F            F            F            M
Age                       38.9         51.0         39.0         39.0
Annual Income (k$)        57.1         79.0         53.1         42.4
Spending Score (1-100)    42.6         30.0         48.0         43.0

Time taken to build model (full training data) : 0.02 seconds

=== Model and evaluation on training set ===

Clustered Instances

0    382        (38%)
1    280        (28%)
2    338        (34%)
```

Fig 1.10 : Hierarchical clustering through Python